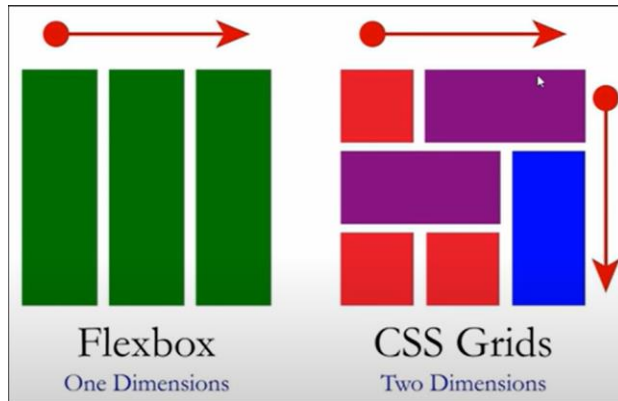


Preparation before W5-S1

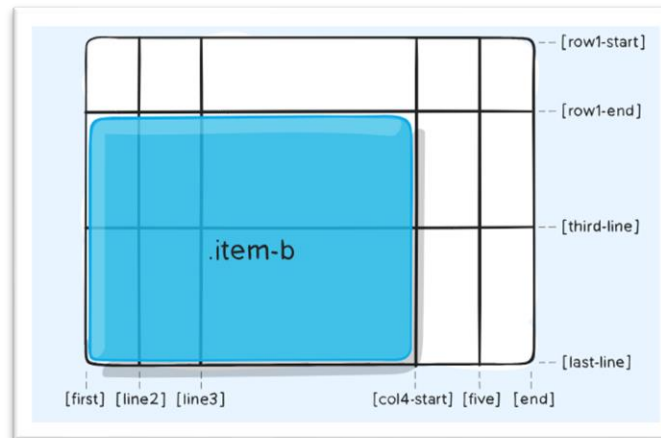
WATCH

- ✓ [Watch this 20min video](#)



LEARN

- ✓ [Read our amazing book](#)
- ✓ [Check the Grid handout](#)



PLAY

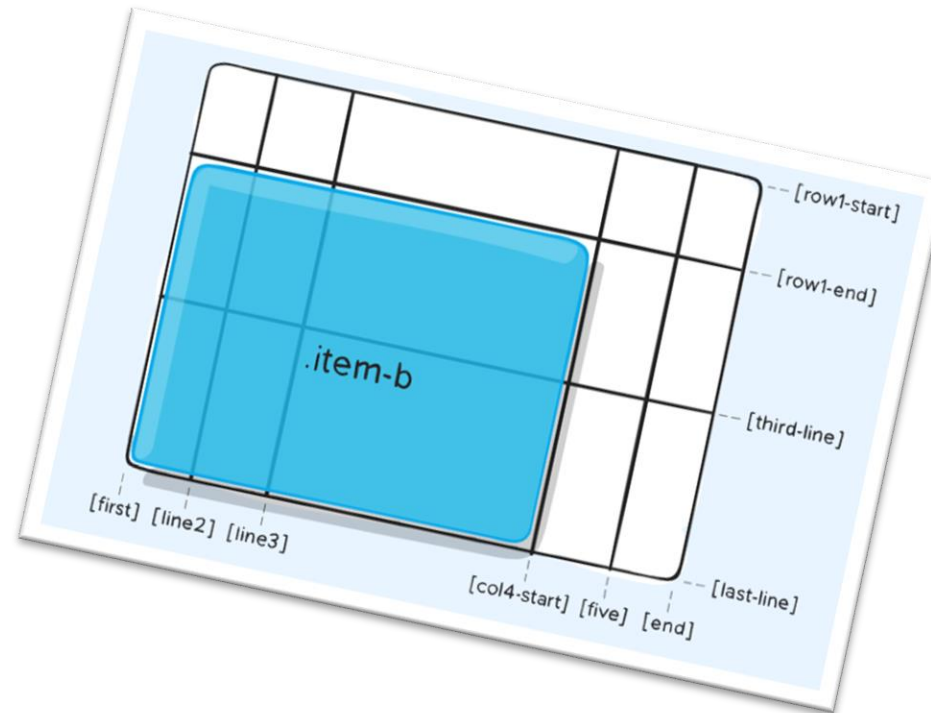
Go as far as you can on this game

<https://cssgridgarden.com>



WEB DESIGN

W5-S1 – GRID LAYOUT





Grid Layout– Objectives



GRID CONTAINER

- ✓ Create a **container**
- ✓ Specify container **column** and **rows**
- ✓ Control all items **alignments**
- ✓ Control **gap** between grid items

GRID ITEMS

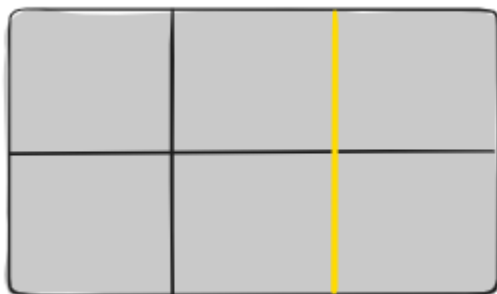
- ✓ **Position an item** within the grid
- ✓ Overwrite a grid **item alignment**

Grid terminology

In this session we will use the following concepts

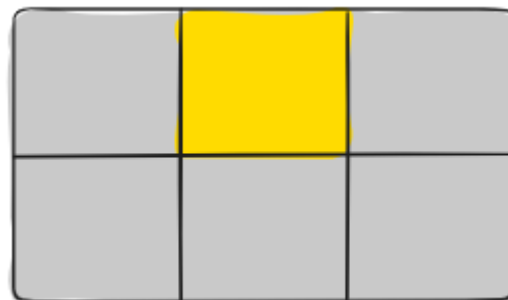
Grid Line

The dividing lines that make up the structure of the grid



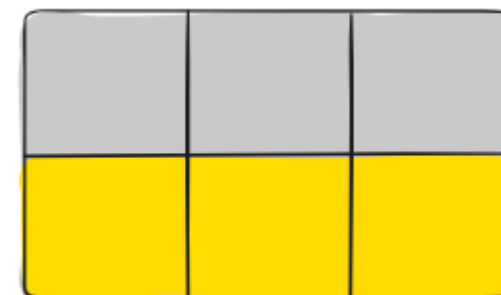
Grid Cell

A single “unit” of the grid



Grid Track

The columns or rows of the grid

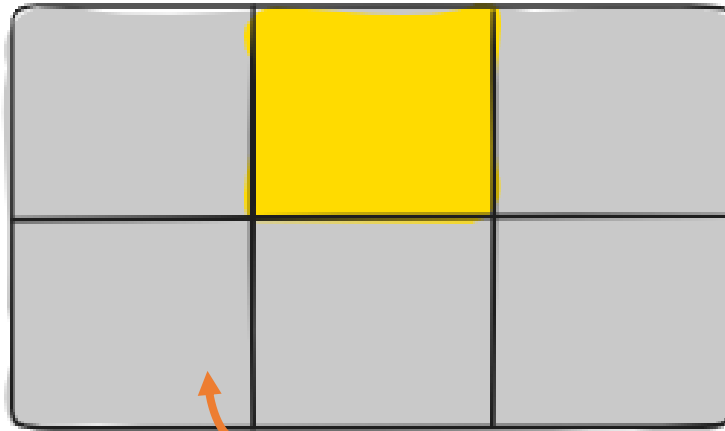


Create a container

Make a difference between grid containers and items

Grid Container

The element on which display: grid is applied



Grid Items

*Grid are the **direct children** of the grid container*

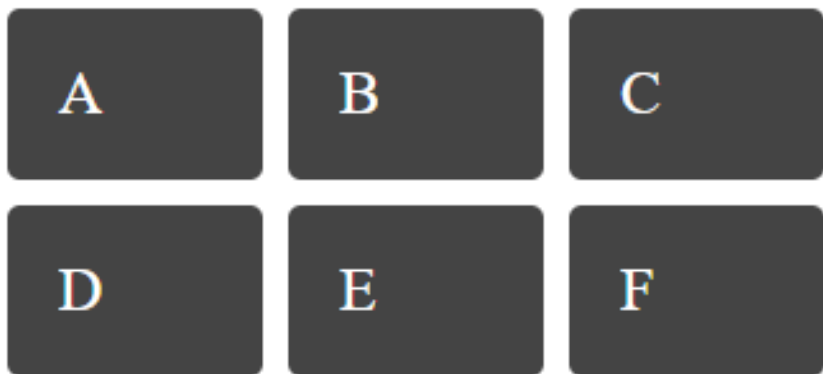
```
<div class="container">  
  <div>Item 1</div>  
  <div>Item 2</div>  
  <div>Item 3</div>  
</div>
```

```
.container {  
  display: grid;  
}
```

Specify column and rows

- **grid-template-columns** Defines the columns of the grid
- **grid-template-rows** Defines the rows of the grid

✓ Check the [code](#) to understand the columns definition



```
<div class="wrapper">
  <div class="box a">A</div>
  <div class="box b">B</div>
  <div class="box c">C</div>
  <div class="box d">D</div>
  <div class="box e">E</div>
  <div class="box f">F</div>
</div>

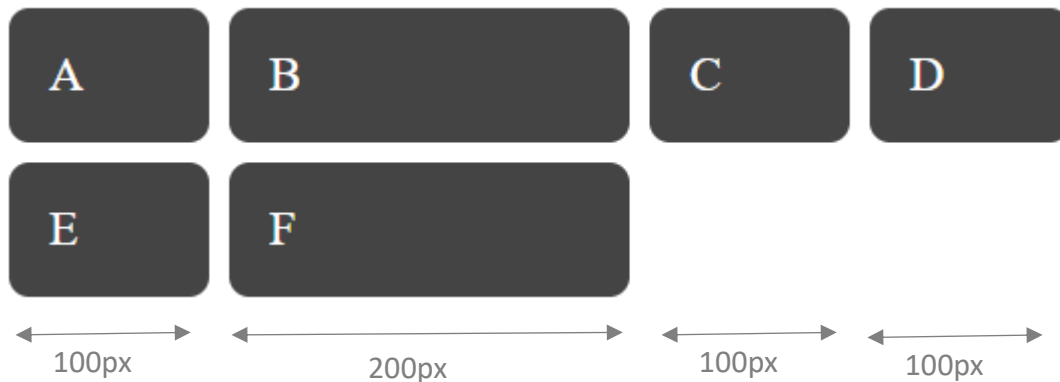
.container {
  display: grid;
  grid-template-columns: 100px 100px 100px;
}
```

Activity 1

- **grid-template-columns**

Defines the columns of the grid

✓ Change the [code](#) to follow the bellow requirements



```
<div class="wrapper">
  <div class="box a">A</div>
  <div class="box b">B</div>
  <div class="box c">C</div>
  <div class="box d">D</div>
  <div class="box e">E</div>
  <div class="box f">F</div>
</div>
```

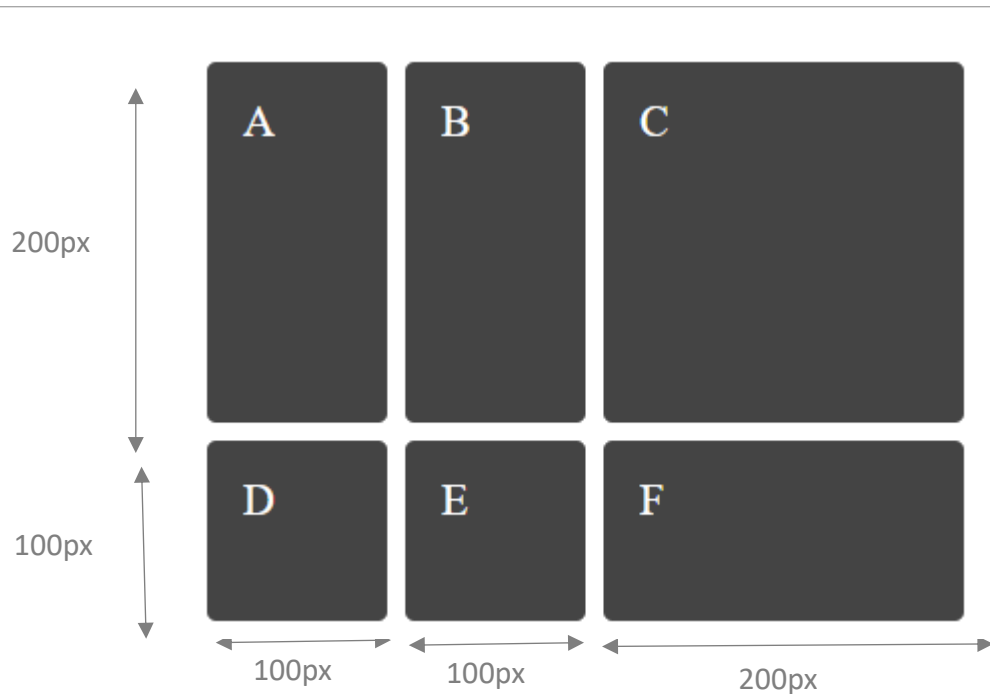
```
.container {
  display: grid;
  grid-template-columns: ??;
}
```

Activity 2

- **grid-template-rows**

Defines the rows of the grid

✓ Change the [code](#) to follow the bellow requirements



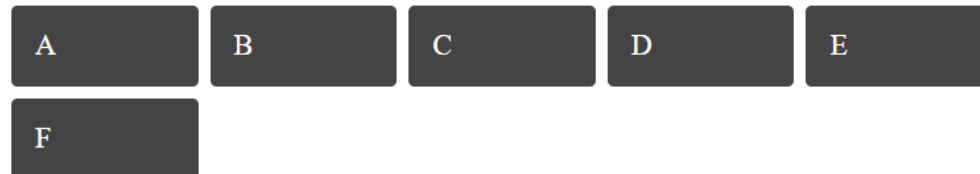
```
<div class="wrapper">
  <div class="box a">A</div>
  <div class="box b">B</div>
  <div class="box c">C</div>
  <div class="box d">D</div>
  <div class="box e">E</div>
  <div class="box f">F</div>
</div>

.container {
  display: grid;
  grid-template-columns: ??;
  grid-template-rows: ??;
}
```


Columns/rows units

Tracks can be specified different ways

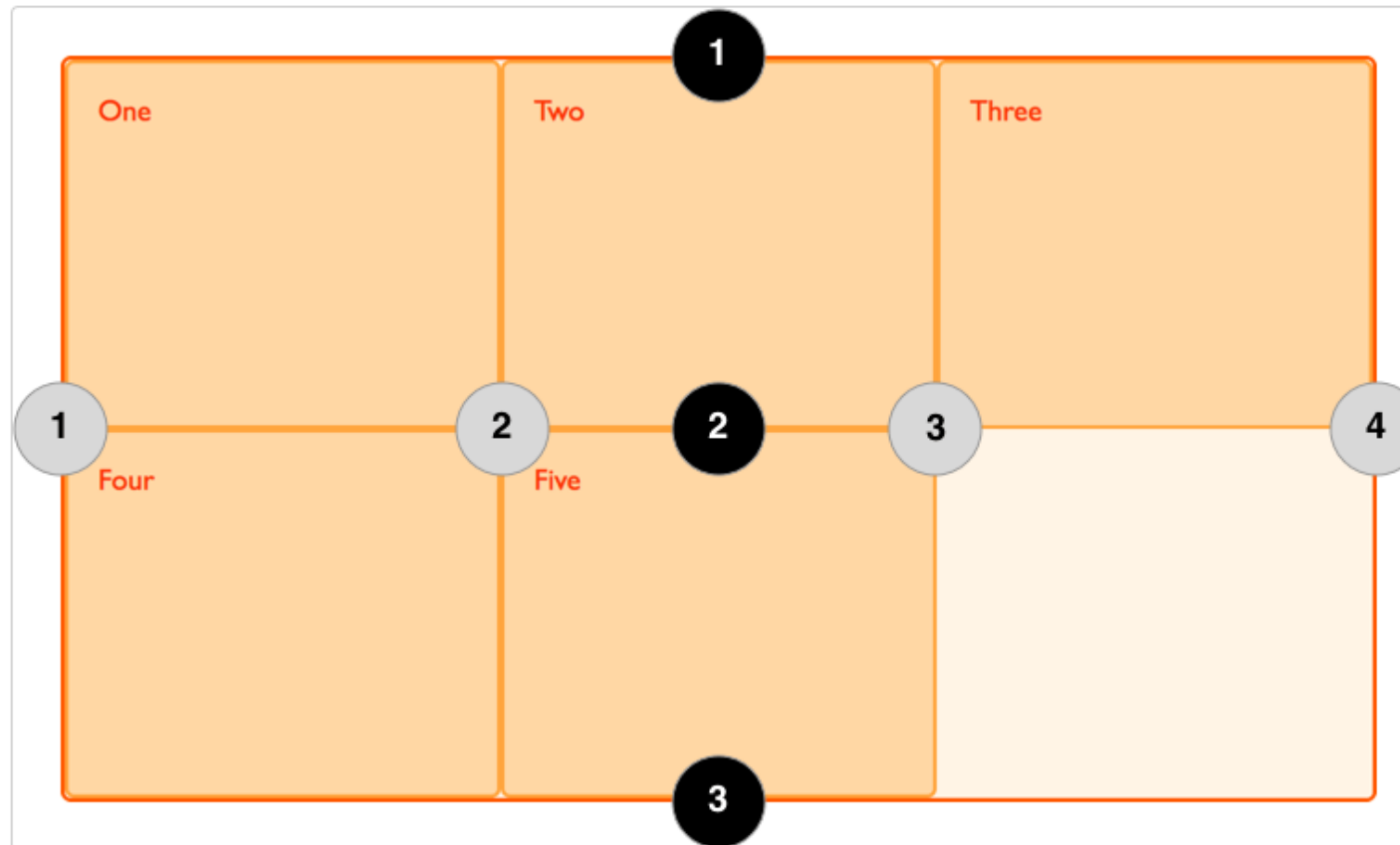
100px 100px	Using pixels
1fr 0.5fr 1fr	Fraction of the available space in the grid container
repeat(5, 1fr)	Repeat 5 times 1fr
Auto	Use the remaining space
50px auto 100px 1fr	<i>We can mix different units...</i>



grid-template-columns: **repeat(5, 1fr);**

Column and Row Lines

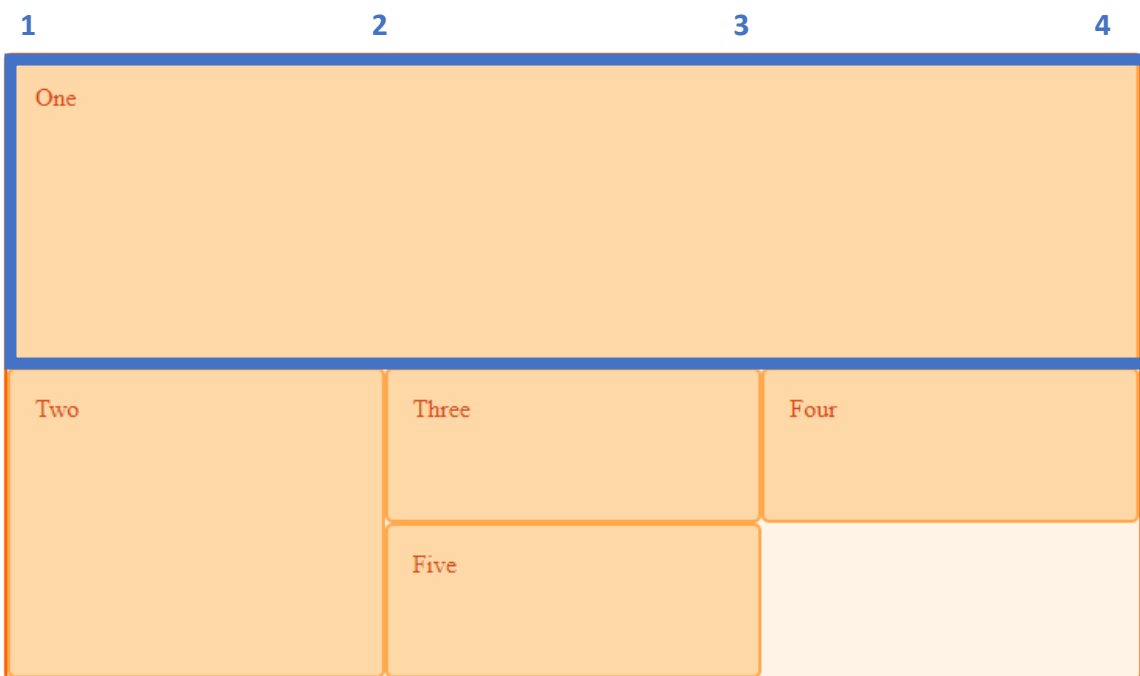
- ✓ When we define a grid we define the grid tracks, not the lines...
- ✓ But Grid then gives us numbered lines to use when **positioning items**.



Position an item within the grid

```
grid-column: 1/4;
```

*Item will start at column **line 1** and will end at **line 4***



```
.wrapper {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
}
```

```
.one {  
  grid-column: 1/4;  
}
```

Position an item within the grid

- ✓ Check the [code](#) to understand the item positioning with lines



```
.wrapper {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
}
```

```
.a {  
  grid-column: 1 / 3; ← A will span on 2 columns  
  grid-row: 1;  
}
```

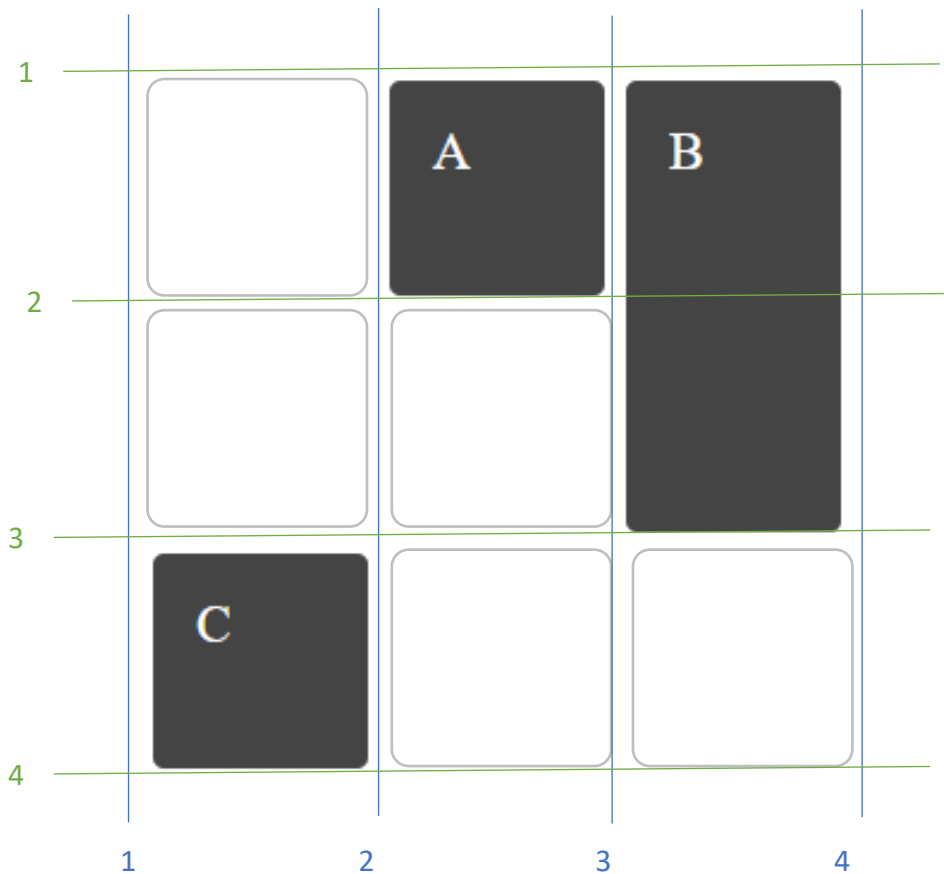
```
.b {  
  grid-column: 3 ;  
  grid-row: 1 / 3; ← B will span on 2 rows  
}
```

```
.c {  
  grid-column: 1 ; ← C start at column line 1  
  grid-row: 2 ;  
}
```

```
.d {  
  grid-column: 2;  
  grid-row: 2;  
}
```

Activity 3

✓ Fil up the ?? to match with bellow image



```
.wrapper {  
  display: grid;  
  grid-template-columns: 100px 100px 100px;  
  grid-template-rows: 100px 100px 100px;  
}  
  
.a {  
  grid-column: 2/3;  
  grid-row: 1/2;  
}  
  
.b {  
  grid-column: ??;  
  grid-row: ??;  
}  
  
.c {  
  grid-column: ??;  
  grid-row: ??;  
}
```

Positioning items against lines + spans

*We can also use the **span** rather than targeting grid Line.*

- ✓ Check the [code](#) to understand the item positioning with lines and spans



```
.a {  
  grid-column: 1 / span 2;  
}
```

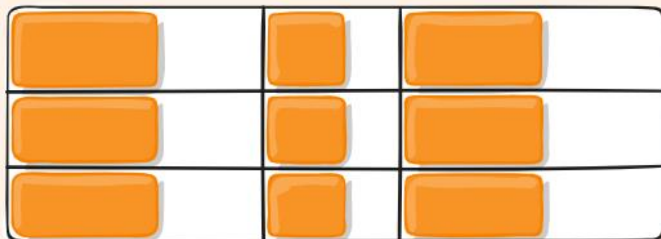
span on 2 columns

Align items and content

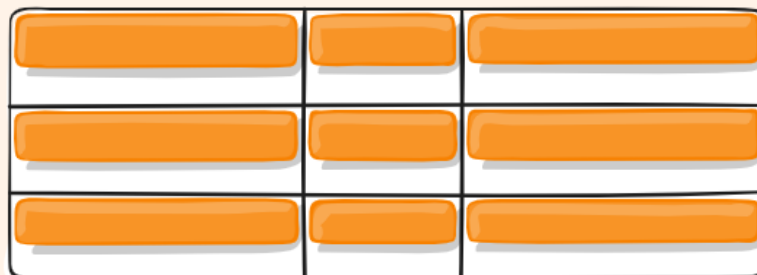
- **justify-items** Aligns items along the row axis (*horizontal*)
- **align-items** Aligns items along the column axis (*vertical*)

- **justify-content** Aligns grid along the row axis (*horizontal*)
- **align-content** Aligns grid along the column axis (*vertical*)

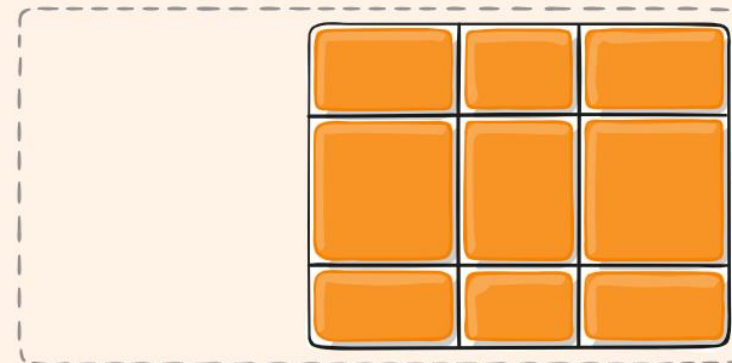
```
.container {  
  justify-items: start;  
}
```



```
.container {  
  align-items: start;  
}
```



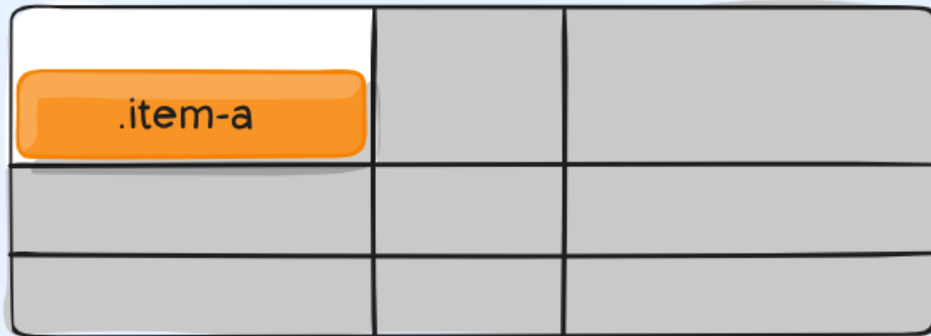
```
.container {  
  justify-content: end;  
}
```



✓ Check the [code](#) to understand the item positioning with lines

Align a single item

- **justify-self** Aligns the item along the row axis (*horizontal*)
- **align-self** Aligns the item along the column axis (*vertical*)



```
.item-a {  
  align-self: end;  
}
```

CSS

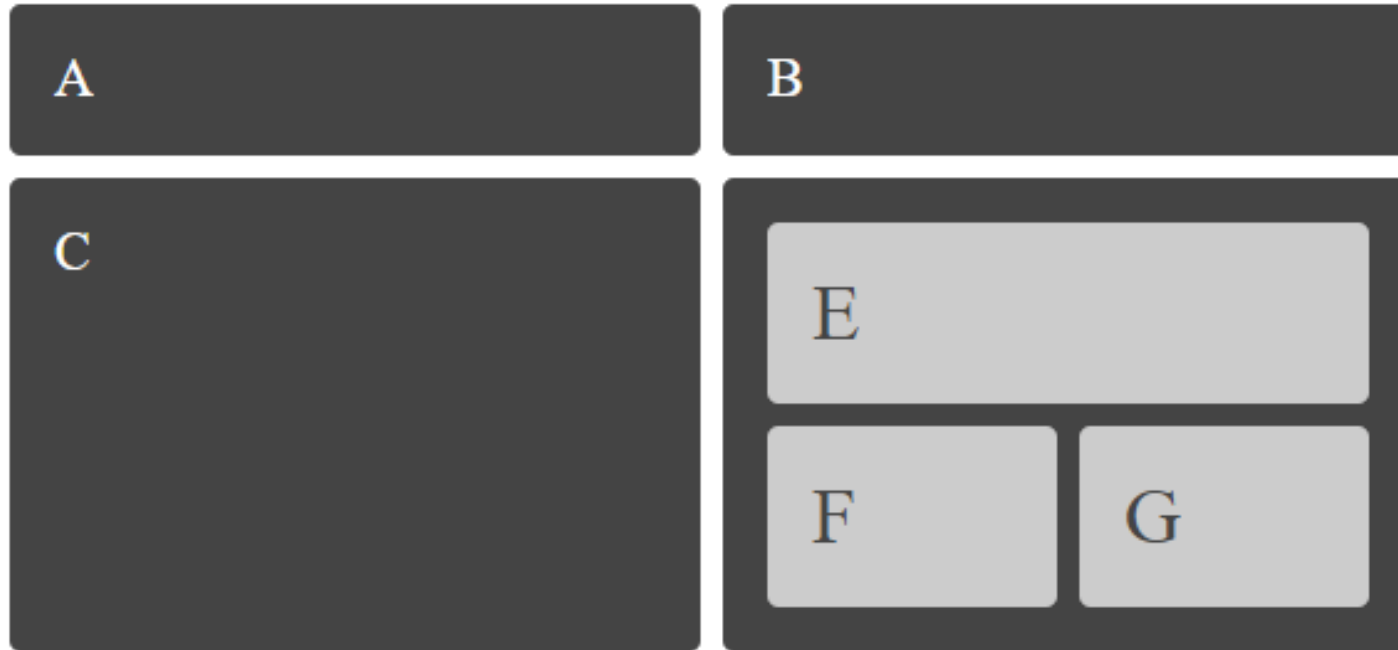
✓ Check the [code](#) to understand the item positioning with lines

Nested Grid

Any Grid Area can become a grid itself...

- By setting `display:grid`
- By defining the rows and columns.

✓ Check the [code](#) to understand the following example



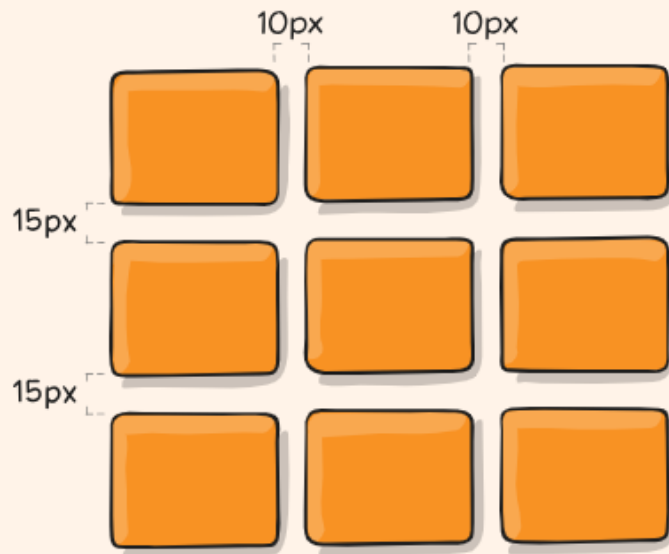
Control gap between grid items

- **row-gap**
- **column-gap**
- **gap**

<row gap>

<column gap>

<row gap> <column gap>



```
.container {  
  grid-template-columns: 100px 50px 100px;  
  grid-template-rows: 80px auto 80px;  
  column-gap: 10px;  
  row-gap: 15px;  
}
```

Or `gap: 15px 10px;`

Use your Browser tool

The diagram on the left illustrates a card layout with five rows. The first row is a 300x200 image. The second row is the title. The third row is the description, which is highlighted with a red double-headed arrow and labeled '1fr', indicating it takes up one full row of space. The fourth row is a dashed line, and the fifth row is the action button.

The HTML code in the middle panel shows the structure of the card. The main container is a grid. The description is wrapped in a `div class="card" span="2"`, which is then wrapped in a `div class="card-container"`.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
    =1.0">
    <title>Boats Article</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="card-container">
      <div class="card span2">
        
        <h2 class="card-title">Card Title</h2>
        <p class="card-description">
        This is a short
        description about
        the card. It
        explains what the
        card is about in a
        few sentences.
        </p>
        <button class="card-button">Action</button>
      </div>
    </main>
  </body>
</html>
```

The CSS code on the right panel shows the styles for the card. The `grid-template-rows` property is set to `auto auto 1fr auto;`, where `1fr` is circled in red, indicating it takes up the most space. The card has a white background, a border radius of 8px, a box shadow, and a padding of 16px.

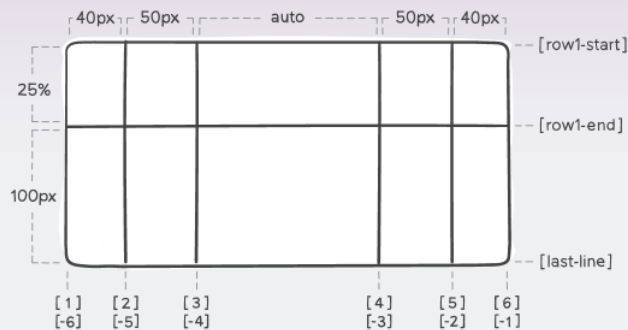
```
.card {
  display: grid;
  grid-template-rows: auto auto 1fr auto;
  background-color: #fff;
  border-radius: 8px;
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
  overflow: hidden;
  gap: 16px;
  padding: 16px;
}
```

Here the card description takes the most row space as it is defined with a 1fr

CSS Grid

```
.container {
  display: grid; /* or inline-grid */
}
```

grid-template-columns
grid-template-rows



```
.container {
  grid-template-columns: <value> | <name>;
  grid-template-rows: <value> | <name>;
}
```

justify-items

start



end



center

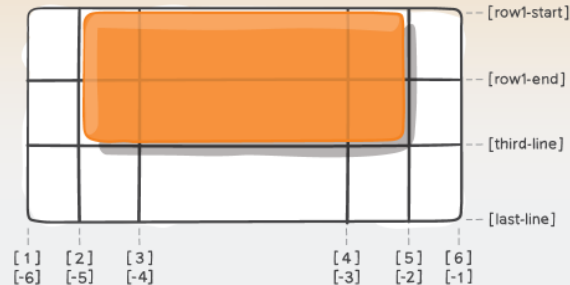


stretch



```
.container {
  justify-items: start | end |
  center | stretch;
}
```

grid-column, grid-row



```
.item {
  grid-column: <start-line> / <end-line> |
  <start-line> / span <value>;
  grid-row: <start-line> / <end-line> |
  <start-line> / span <value>;
}
```

justify-content

start



end



center



stretch



space-around

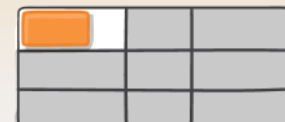


space-between

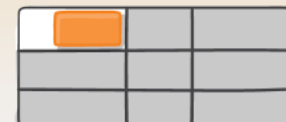


justify-self

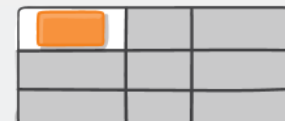
start



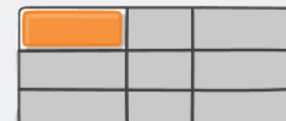
end



center



stretch



```
.item {
  justify-self: start | end | center |
  stretch;
}
```

Let's review the 5 last Grid Garden challenges!

<https://cssgridgarden.com>



For each challenge, 1 student explain his/her solution

After this session

1 REVIEW

- ✓ [Video](#)
- ✓ [Mozilla Docs](#)
- ✓ [Mozilla Docs Part 2](#)
- ✓ [CSS Grid Handout](#)

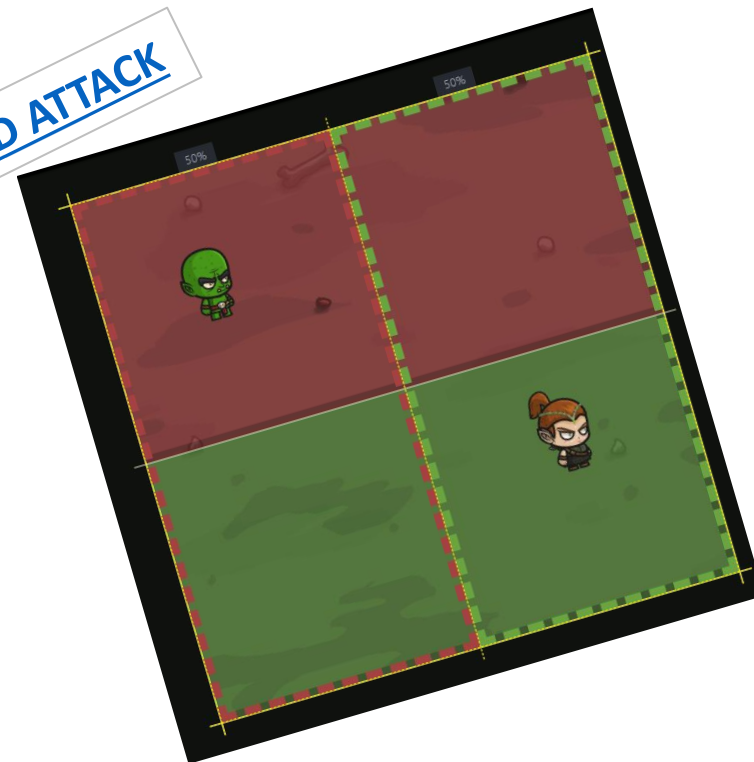
[GET THE POSTER](#)



2 – PLAY !

<https://codingfantasy.com/games/css-grid-attack>

[PLAY GRID ATTACK](#)



3-2-1 Challenge

- ✓ List three things you **learned** today.
- ✓ List two **questions** you still have.
- ✓ List one aspect of the lesson or topic you **enjoyed**.

